UniDPP

DIGITAL PRODUCT PASSPORT · MULTI-BODY STEWARDSHIP

UNIDPP CONTRIBUTION

ISO/IEC JTC 5 · DIGITAL PRODUCT PASSPORT

UniDPP: a reference implementation of the international framework

---

2026-11-08
UniDPP — convened under CalConnect, multi-body stewardship

01 · SCOPE AND STATUS

Why an implementation report

None of the eight ENs of the DPP series contains a conformance clause: the published European standards specify less about what an implementation must do to claim conformity than the consortium specifications beside them (IDTA 01002, for example, ships a machine-validated JSON Schema and a conformance corpus for its API). ISO values implementation experience precisely because it exposes the difference between what a text says and what a system can do. This paper reports the UniDPP project: an Apache-2.0/MIT multi-repository implementation of the framework described in paper 1, built to run the EN series as one jurisdiction profile among several.

Honest status marking is used throughout: **[built]** components exist and run; **[in progress]** components are under construction with a committed design; **[planned]** components have a specification and a roadmap slot. Nothing on this page claims production readiness.

02 · ARCHITECTURE

The repository map

The implementation follows the framework's layering (paper 1, §04): a neutral core in Rust, generated bindings for the web, registry and terminology integrations in Ruby, verification harnesses in Python, and the trust layer contributed by the SIGNATIF stack.

Repository	Contents	Status	Tests
unidpp-core (Rust)	Scheme-agnostic identity with ISO/IEC 15459 parsing and bridging; core passport model with manifests and typed PassportLink edges; typed events over an append-only log with commitment hashing; split/combine transforms with mass balance and taint propagation; the Tier-A offline packer (minimum fields, multi-suite signature framing, carrier-size budgeting); graded trust verdicts with coverage reports and freshness; resolver linkset emission/parsing with context routing; the unidpp-demo CLI replaying three narrated scenarios (battery loop, car, laptop) with byte-exact traces. Property tests: quantity conservation, commitment consistency, casing-normalization conformance.	[built]	118
unidpp-spec	JSON Schema for the core model and profiles; the single source for generated types; profile templates.	[in progress]	—
unidpp-ts	Types generated from unidpp-spec; client libraries for resolver, registry and passport APIs; validators; browser-side Tier-A verification. Published as `@unidpp/*`.	[planned]	—
unidpp-rb	Registry client (FERIN/19135 items, statuses, supersession); glossarist bridge (terminology ↔ registry anchors); expressir/CDDAL hooks for schema validation; CNML/ESDC handling via lutaml-model. Published as gem `unidpp`.	[built]	132
unidpp-py	Verification-pipeline harness driving unidpp-core; conformance runners (positive corpus plus audit-derived negative cases — the EU-profile runner of §05 among them); adapters for UNTP, EPCIS and GS1 Digital Link. Published as `unidpp` on PyPI.	[built]	133
unidpp-resolver	Linkset resolver with profile-context routing (RFC 9264 linksets), carrier→identifier normalization (GS1 Digital Link, GB/T 33993, legacy EAN-13, ISO/IEC 15459 URN), dark identities, as-of stamped resolution and national-intermediary mode.	[built]	44

unidpp-signatif	Trust layer over the SIGNATIF model: real multi-suite signatures (Ed25519, ECDSA P-256), delegation trust graph with scope enforcement, revocation whose reason determines retroactivity, transparency anchoring, graded verification; threshold quorum operations through the Confium interface seam (binding deferred).	[built]	71
unidpp-explorer	Developer tooling for inspecting passports, event logs and profiles; browser-side Tier-A verification with live WebCrypto.	[built]	browser
unidpp.github.io	Documentation site (www.unidpp.org): framework pages, capability comparison, terminology browser embed.	[in progress]	—

The website and its progressive-disclosure information architecture

The documentation site is deliberately not a flat namespace of technical pages. It is arranged as one depth sequence — **read by depth**, where each level assumes the one before it: a plain-language account of what a passport is (told with a washing machine); the international case (one laptop, five jurisdictions, one record instead of five); the mechanism (the six layers, the fourteen invariants, capability classes, trust); and only then the full reference material (the framework in full, the standards map, the implementation, the comparisons, the terminology, the papers, and stewardship). A market-surveillance officer, a committee delegate and a software engineer each enter at their own depth without being forced through material they do not need. The **terminology page deploys the Glossarist Concept Browser edition** of the dataset announced in paper 6 (glossarist.org/dpp), so the vocabulary the framework proposes is inspectable from the same site that documents the implementation that exercises it.

Conformance approach

The eight ENs are implemented together as a single jurisdiction profile — the "EU profile" — over the neutral core, exactly as the framework prescribes for any legal regime: EN 18223's data model maps to profile-declared data points, EN 18222's API to the profile's served interface, EN 18239's roles to the profile's access policies, EN 18220's carriers and EN 18216's transport to the profile's carrier and transport bindings. The core remains jurisdiction-free; nothing EU-specific leaks below the profile line (invariant I11 checked mechanically: the core test suite forbids EU-profile imports).

Conformance is corpus-driven. The positive corpus is assembled from every worked example in the EN texts and the IDTA mapping tables (notably IDTA 02099-1, the submodel-to-EN mapping template). The negative corpus is derived from the audit register of corpus defects: each defect class becomes a validator test that must reject the defective form. This is the **conformance fixture plan**: every item of the defect register (AUDIT) becomes a negative fixture with its citation attached, so the corpus's own defects define the rejection suite — the published texts are the richest available source of inputs a conforming implementation must refuse. Two consequences follow. First, casing and format drift become *normalization conformance* tests rather than editorial complaints: the implementation accepts `"granularity": "Model"` and `"model"` (the EN prints the lowercase enumeration in §4.1.2.2 and the capitalized form in its own §5.2.4 example), normalizes, and records which form was seen. Second, timestamp validation is strict ISO 8601-1 — the corpus contains a published example using en-dash separators, which the validator must reject and the report must log. The plan is no longer prospective: the run over the complete EN 18223 example corpus has been executed, and section 05 reports its results.

Methodology and a worked example

Drift findings are produced by a fixed method: (1) convert the source documents (the eight ENs, two CWAs, 77 IDTA specifications, AWI 25534-1) to a common machine-readable form; (2) extract identifier vocabulary, enumerations and examples; (3) cross-reference against the claims each document makes about the others; (4) classify divergences as inconsistency, underspecification or editorial; (5) encode each finding as a test. The method is reproducible from the published texts, and every finding carries a clause-level citation.

> **Worked example — `lastUpdated` vs `lastUpdate`.** EN 18223 names the passport update timestamp `lastUpdated`. IDTA 02099-1, the consortium specification whose purpose is to map its submodel content into the EN's data model, names the corresponding field `lastUpdate`. An implementer following both documents cannot tell which spelling the registry will index; a casing-normalization layer must be introduced on one side or the other, and neither document says whose job that is. In UniDPP the finding becomes a conformance fixture: both spellings are accepted at the EU-profile boundary, normalized to the core's canonical field, and the acceptance is recorded in the verification coverage report — the drift is visible in every verdict instead of silent. The same machinery covers the *Model/model* and *Active/active* divergences between the same two documents.

The full defect register (classification, citations, and the test each item became) is published alongside the implementation and offered to JTC 5 as standing evidence for the conformance workstream: the series' own texts are the richest available source of negative test cases. In framework terms (paper 1, §03), every item in the register is a seam failure — a defect at one of the twelve layer joins — and encoding each item as a test pins the seam it broke.

05 · RESULTS

The conformance run and the demonstrations

The EN 18223 example corpus, machine-run

The fixture plan of section 03 has been executed against EN 18223:2026 (final text, corrected and reissued 2 June 2026). Every example payload printed in the standard — **22 fixtures**: the eight compressed JSON fragments of clause 5.2, the six expanded JSON examples of Annex A, and the eight XML examples of Annex B — was extracted verbatim and run through the validators, each finding cited to the clause it violates. Outcome: **14 pass, 8 fail, 26 error-severity findings**. The distribution is the result: **all six Annex A examples — the normative expanded-JSON serialization the standard teaches — fail**, together with the clause 5.2.4 header fragment and Annex B Example 1 (which carry the same four header defects); the seven clause 5.2 data-element fragments and Annex B Examples 2–8 pass.

Fixture	Clause	Outcome	Findings
cl5-5.2.4-header	5.2.4	fail	A1, A2, A6, C4
cl5-5.2.5-dataelementcollection	5.2.5	pass	—
cl5-5.2.6-singlevalued-standalone	5.2.6	pass	—
cl5-5.2.6-singlevalued-in-complex	5.2.6	pass	—
cl5-5.2.7-multivalued-native	5.2.7	pass	—
cl5-5.2.7-multivalued-objects	5.2.7	pass	—
cl5-5.2.8-relatedresource	5.2.8	pass	—
cl5-5.2.9-multilanguage	5.2.9	pass	—
annexA-example1	Annex A, Ex. 1	fail	A3 (×4), C1
annexA-example2	Annex A, Ex. 2	fail	N1
annexA-example3	Annex A, Ex. 3	fail	A3 (×2), N2 (×2)
annexA-example4	Annex A, Ex. 4	fail	N5, N3, A3 (×3), N4 (×3), C1
annexA-example5	Annex A, Ex. 5	fail	N6, A5

annexA-example6	Annex A, Ex. 6	fail	A5
annexB-example1	Annex B, Ex. 1	fail	A1, A2, A6, C4
annexB-example2 … example8	Annex B, Ex. 2–8	pass ×7	—

Findings cross-reference the defect register (AUDIT): the pre-registered classes A1–A6 (granularity and status casing, class-name drift, timestamp form, language-tag structure, schema-version placeholder) and B2 (the IDTA `lastUpdate` spelling, exercised as a probe), plus the vacancy references C1 and C4, which do not affect the outcome. The run also produced **six new findings, N1–N6**, each a cited fact of the EN's own text:

ID	Location	Defect
N1	Annex A, Ex. 2	Not well-formed JSON: misnested/unclosed braces; literal ellipsis array placeholders in a normative annex.
N2	Annex A, Ex. 3	`dictionaryReference` `https:/dictionary1/...` omits the `//` authority separator.
N3	Annex A, Ex. 4	Duplicate elementId `efficiencyRating2` — violates the Table 2 uniqueness requirement.
N4	Annex A, Ex. 4	`xsd:float` values serialized as JSON strings, contradicting Table 7 and the §5.2.5 compressed numeric form.
N5	Annex A, Ex. 4 vs Ex. 1	MultiValuedDataElement children keyed `value` vs `elements` — two serializations of one class.
N6	Annex A, Ex. 5	Trailing comma — invalid JSON in a normative annex.

Mapping the EU compressed and expanded forms onto the neutral core required **nine documented profile adaptations, PA1–PA9**: JSON line-comment stripping (the clause 5.2 examples print `//` comments inside payloads); the `lastUpdate` ↔ `lastUpdated` key mapping; granularity and status value-case normalization; compressed → expanded promotion (the EN specifies the two forms by example and defines no transformation); positional elementId synthesis for anonymous array items; MultiValuedDataElement child-key tolerance; XML projection with inverse typing of scalar text; fragment tolerance for the §5.2 extracts; and URI-formatted product identifiers mapped onto the core's identifier model. Every adaptation is an engineering accommodation of an underspecified or internally inconsistent text, recorded rather than improvised — the count is itself a measure of the distance between the EN's two serializations. The full report, with every finding's verbatim citation, is published as the run's evidence annex together with the fixtures and the runner, so the table above can be reproduced from the purchased text (source licence constraint: the fixtures are not redistributed).

The demonstrations: byte-exact traces

The second body of implementation evidence is the `unidpp-demo` CLI, which replays three narrated scenarios end-to-end: the battery loop (combine, split, blind installation, theft taint propagating through the provenance DAG, end-of-waste, Tier-A packing, graded verdicts), the car (passport federation, predicate-based recall, as-of readings then and now), and the laptop (one neutral core under two jurisdiction lenses). Every timestamp is fixed and every salt is derived deterministically from the run seed; the workspace test suite executes each scenario twice and asserts **byte-identical output (118/118 workspace tests green)** — a printed trace is also a passing conformance run. The narrated walkthroughs are published with run instructions.

The Tier-A byte numbers are the headline. The battery pack's offline payload projects to **405 B against a QR version 15 EC-M capacity of 412 B — margin +7 B**, with both signature slots framed (ECDSA P-256 and SM2, so an EU verifier and a CN verifier each check under their own crypto policy). Substituting ML-DSA-65 for the two classical suites projects to **3570 B against the 2331 B ceiling** at version 40 — rejected with an over-budget error, never silently truncated: the offline minimum is fixed membership, not an elastic field. And the laptop scenario produces the same-passport, two-lens result the framework predicts — **PASS under the EU lens (coverage 3/3), DEGRADED under the JP lens (coverage 2/3)**, naming the missing data point — inter-profile disagreement as an explicit, auditable outcome rather than a silent merge.

06 · BENCHMARKS PLANNED

freeDPP and OpenDPP, factually

Two open implementations already exist, and the committee should read them as data, not as competition. **freeDPP** (German camp; author Otto Handle, CEN-CLC/JTC 24/WG 4 convenor) is a minimal ASP.NET server publishing DPPs as JSON per the standards and an HTML view, explicitly "not yet for production", with the eight ENs as its entire requirement set. **OpenDPP** (international camp, closed core) is an AAS-conformant B2B platform whose most neutral feature is UNTP coverage, with an interop repository carrying conformance badges (IDTA AAS v3.0/3.1, UNTP DPP v0.7.0, GS1 Digital Link, W3C VC), a machine-readable knowledge projection of its integration API, and generated SDKs version-locked to its OpenAPI contract.

The capability matrix below is the factual basis for the planned benchmark suite. It is compiled from public repositories and documentation of both projects (assessed 2026-09) and from UniDPP's own roadmap; it contains no capability UniDPP has not either demonstrated or specified. The benchmark suite itself is [planned]: identical workloads (issuance, update event append, full verification, offline Tier-A verification, registry-dependent validation) executed against all three implementations where the capability exists, with published methodology and raw results.

Capability	freeDPP	OpenDPP	UniDPP
EN 182xx (EU profile)	Yes	Yes	Yes, as one profile
Multi-jurisdiction profiles	No	No	Core mechanism [built]
Neutral registry (FERIN/19135)	No	No	Designed; client [built]
Trust: multi-suite + transparency logs	No (TLS only)	W3C VC only	SIGNATIF integration [built]; Confium [planned]
Event sourcing + transformation algebra	No	No	Core events [built]
Offline Tier A	No	No	Packer [built]
AI-agent knowledge projection	No	Yes (from private backend)	From the open specification [planned]
Terminology dataset (multilingual)	No	No	Glossarist dataset [built]
Conformance suites published	Partial (none published)	CONFORMANCE.md	Full, open [built] — §05 results

Licence	GPL-3 server	Apache-2 boundary, closed core	Fully open (Apache-2/MIT per repo)

Reading of the matrix, stated without evaluative language: freeDPP demonstrates that the eight ENs alone yield a single-jurisdiction system by construction — its README treats them as the entire requirement set. OpenDPP demonstrates that AI-agent projections and conformance badges can be built — from a private backend, with a closed core. Neither addresses the profile mechanism, the registry vacancy, the event model or offline verification, which is where UniDPP concentrates. freeDPP's sample payloads are imported into UniDPP's EU-profile fixture set, crediting the source.

07 · REFERENCES

1. EN 18216–18246 series; EN 18223:2026 (final text, corrected and reissued 2 June 2026); IDTA 02099-1 (submodel-to-EN mapping template).
2. UniDPP EU-profile conformance report (evidence annex: 22 fixtures, per-fixture findings with verbatim citations, N1–N6 register, PA1–PA9 adaptations) and the unidpp-demo narrated traces — both published with the implementation.
3. freeDPP repositories (freeDPP, freeDPPserver, freeDPPdatabase, freeDPPgui), public, assessed 2026-09.
4. OpenDPP repositories (opendpp-interop, opendpp-knowledge, opendpp-sdk), public, assessed 2026-09.
5. SIGNATIF (signatif.org); Confium; Glossarist; UNTP (UNECE).
6. Companion papers 1, 2, 3 and 5 of this series.

UniDPP project, convened under CalConnect — multi-body stewardship; Apache-2.0/MIT per repository.

Paper 4 of 6 · unidpp-reference-implementation.html · 2026-09-07